

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Cryptography and Network Security

COURSE CODE: CSA51

COURSE OUTCOME COVERED

CO3: *Examine cryptographic hash algorithms, digital signature schemes, and assess Kerberos and X.509 authentication frameworks for ensuring integrity, authentication, and non-repudiation.CO (BL4 , BL5)*

Assessment Tool Weightage: 40%

QUESTIONS: 5

TOTAL MARKS: 25

EXPERIMENTS

Code of Conduct:

I T Sanhith(192311228) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A Experiments

1. Implement and compare MD5 and Secure Hash Algorithm (SHA-256) in Python by hashing multiple input messages. Analyze their collision resistance and performance differences.

Feature	MD5	SHA-256
Hash Length	128 bits (32 hex characters)	256 bits (64 hex characters)
Speed	Faster	Slightly slower
Security	Weak	Strong
CollisionResistance	Poor	Excellent
Current Usage	Checksums,nonsecurity purposes	Digital signatures,SSL/TLS, blockchain,password hashing (with salting/KDFs)

```

def md5_hexdigest(msg_bytes):
    md5_obj = hashlib.md5(msg_bytes)
    md5_hex = md5_obj.hexdigest()
    return md5_hex

def sha256_hexdigest(msg_bytes):
    sha256_obj = hashlib.sha256(msg_bytes)
    sha256_hex = sha256_obj.hexdigest()
    return sha256_hex

# Example usage
msg = b'Hello, World!'

md5_hex = md5_hexdigest(msg)
sha256_hex = sha256_hexdigest(msg)

# Display results
print(f"Message: {msg}")
print(f"MD5 Hash: {md5_hex}")
print(f"SHA-256 Hash: {sha256_hex}")

# Performance Comparison
import time

start_time = time.time()
md5_hexdigest(msg)
end_time = time.time()
md5_time = end_time - start_time

start_time = time.time()
sha256_hexdigest(msg)
end_time = time.time()
sha256_time = end_time - start_time

print(f"MD5 Execution Time: {md5_time} s")
print(f"SHA-256 Execution Time: {sha256_time} s")

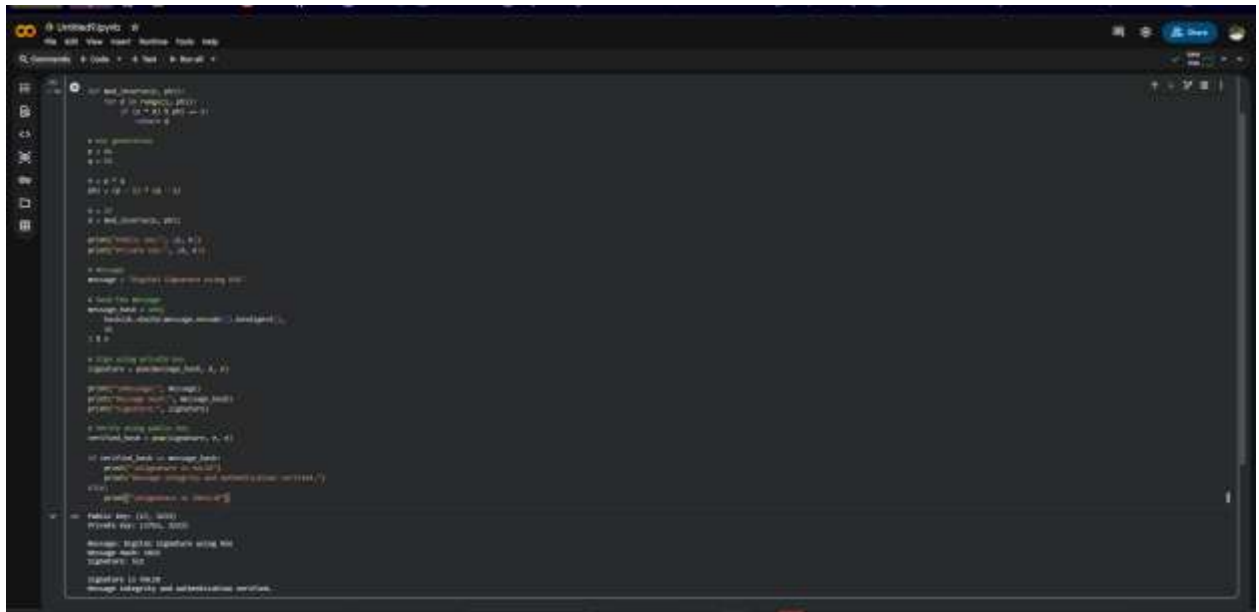
# Collision Resistance Test
def find_collision(md5_func, sha256_func):
    """
    A simple test to find collisions for MD5 and SHA-256.
    This is a placeholder for a more complex collision-finding algorithm.
    """
    pass

# Test for collisions
md5_collision = find_collision(md5_hexdigest, sha256_hexdigest)
sha256_collision = find_collision(sha256_hexdigest, md5_hexdigest)

print(f"MD5 Collision Found: {md5_collision}")
print(f"SHA-256 Collision Found: {sha256_collision}")

```

2. Develop a Python program to implement a Digital Signature scheme using RSA. Demonstrate message signing and signature verification, and analyze its effectiveness in ensuring integrity and authentication.



```
# RSA Digital Signature Scheme
import random
import hashlib
import sys

# RSA Parameters
p = 61
q = 53
n = p * q
phi = (p - 1) * (q - 1)

# Generate Private Key
e = 7
d = pow(e, -1, phi)

# Generate Public Key
pub_key = (n, e)
priv_key = (n, d)

# Message
message = "Digital Signature using RSA"

# Hash the message
hash = hashlib.sha256(message.encode()).hexdigest()

# Sign the message
signature = pow(hash, d, n)

# Print results
print("Message:", message)
print("Public Key:", pub_key)
print("Private Key:", priv_key)
print("Signature:", signature)

# Verify the message
verified_hash = pow(signature, e, n)
print("Verified Hash:", verified_hash)
print("Message Integrity and Authentication verified!")
```

Integrity and Authentication Analysis

Digital signatures provide message integrity because even a small modification to the message produces a different hash value. Therefore, if the message is changed after signing, signature verification will fail.

Authentication is provided because the signature is generated using the sender's private key. Since only the legitimate sender should possess the private key, successful verification using the corresponding public key confirms the authenticity of the sender.

RSA digital signatures also provide non-repudiation because the sender cannot easily deny signing a message if the private key is securely controlled.

3. Write a Python program to simulate HMAC (Hash-based Message Authentication Code) using a chosen hash function. Compare its security properties with a simple hash-based integrity check.

Security Comparison

Feature	Simple Hash	HMAC
Message Integrity	Yes	Yes
Authentication	No	Yes
Uses Secret Key	No	Yes
Detects Unauthorized Modification	Limited	Yes
Suitable for Secure Communication	No	Yes

```

import hashlib
import sys
import random
import os

def generate_secret_key():
    """Generate a random secret key of 32 bytes"""
    return os.urandom(32)

def calculate_hmac(message, key):
    """Calculate HMAC for a message using a secret key"""
    return hashlib.new('sha256', (key + message)).hexdigest()

def verify_hmac(message, key, hmac):
    """Verify the HMAC of a message using a secret key"""
    return calculate_hmac(message, key) == hmac

def main():
    # Generate a secret key
    key = generate_secret_key()

    # Create a message
    message = "This is a secret message"

    # Calculate the HMAC
    hmac = calculate_hmac(message, key)

    # Print the message and HMAC
    print(f"Message: {message}")
    print(f"HMAC: {hmac}")

    # Simulate receiving the message and HMAC
    received_message = message
    received_hmac = hmac

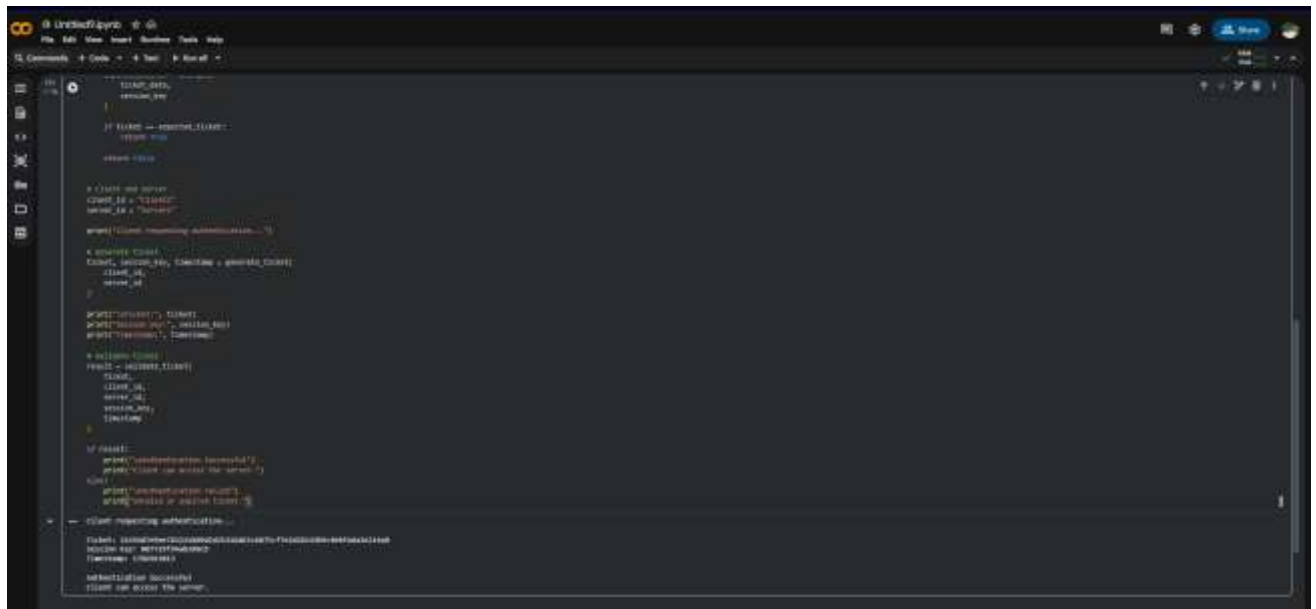
    # Verify the HMAC
    is_valid = verify_hmac(received_message, key, received_hmac)

    # Print the result
    print(f"Is the message valid? {is_valid}")

if __name__ == "__main__":
    main()

```

4. Implement a simplified Kerberos authentication mechanism in Python to simulate ticket generation, distribution, and validation between a client and server. Analyze its effectiveness in preventing replay attacks.



5. Develop a Python-based implementation of either the ElGamal or Schnorr digital signature scheme, and evaluate its security features compared to standard digital signature approaches.

```
import hashlib
```

```
import random
```

```
# Function to find GCD
```

```
def gcd(a, b):
```

```
    while b != 0:
```

```
        a, b = b, a % b
```

```
    return a
```

```
# Function to find modular inverse
```

```
def mod_inverse(a, m):
```

```
    for x in range(1, m):
```

```
        if (a * x) % m == 1:
```

```
            return x
```

```
    return None
```

```
# Public parameters
```

```
p = 467
```

```
g = 2
```

```
# Private key
```

```
x = 127
```

```
# Generate public key
```

```

y = pow(g, x, p)
print("Public Key:", y)
print("Private Key:", x)
# Message
message = "ElGamal Digital Signature"
# Hash message
h = int(
    hashlib.sha256(message.encode()).hexdigest(),
    16
) % (p - 1)
# Generate random k
k = random.randint(2, p - 2)
while gcd(k, p - 1) != 1:
    k = random.randint(2, p - 2)
# Generate signature
r = pow(g, k, p)
k_inverse = mod_inverse(k, p - 1)
s = ((h - x * r) * k_inverse) % (p - 1)
print("\nMessage:", message)
print("Hash:", h)
print("\nSignature:")
print("r =", r)
print("s =", s)
# Verify signature
left_side = pow(g, h, p)
right_side = (
    pow(y, r, p) *
    pow(r, s, p)
) % p
if left_side == right_side:
    print("\nSignature is VALID")
else:
    print("\nSignature is INVALID")

```

```

1 // Example: A simple recursive function to calculate the sum of numbers from 1 to n.
2
3 // Base case: If n is 1, return 1.
4 // Recursive case: Return n plus the sum of numbers from 1 to n-1.
5
6 // Function definition
7 int sum(int n) {
8     // Base case
9     if (n == 1) {
10         return 1;
11     }
12     // Recursive case
13     return n + sum(n - 1);
14 }
15
16 // Main function
17 int main() {
18     // Input
19     int n = 5;
20     // Output
21     cout << "Sum of numbers from 1 to " << n << " is: " << sum(n) << endl;
22     return 0;
23 }

```

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation